

C# Threading & Concurrency – Training Curriculum

Prerequisite Primer (before Phase 1)

#	Topic	Subtopics	Notes
0	C# Language Prerequisites	Delegates; Anonymous methods (<code>delegate { }</code>); Lambdas; Closures and the loop-variable capture gotcha; <code>Func<></code> and <code>Action<></code>	-

Core Curriculum

#	Phase	Subtopics	Notes
1	Foundations	Process vs thread vs AppDomain; Why threading (responsiveness, parallelism, throughput); Thread lifecycle and states; UI/main thread concept; Foreground vs background threads; Context-switching cost; CPU cores vs threads	Conceptual session. No heavy code. Use diagrams.
2	Low-level Thread API	<code>System.Threading.Thread</code> (Start, Join, Sleep); Passing parameters; <code>ParameterizedThreadStart</code> ; Why <code>Thread.Abort</code> is deprecated; <code>ThreadPool.QueueUserWorkItem</code> ; When (not) to use <code>Thread</code> directly	Teach as historical baseline. Most modern code uses <code>Task</code> .
3	Synchronization Primitives	Race conditions and atomicity; <code>Lock</code> / <code>Monitor</code> ; Picking a good lock object; <code>Mutex</code> , <code>Semaphore</code> , <code>SemaphoreSlim</code> ; <code>ReaderWriterLockSlim</code> ; <code>Interlocked</code> (Increment, CompareExchange); <code>volatile</code> and memory barriers (briefly); <code>ManualResetEvent(Slim)</code> , <code>AutoResetEvent</code> , <code>CountdownEvent</code> ; Deadlock prevention (lock ordering, timeouts)	The hardest phase. Plan a deadlock-debugging exercise.
4	BackgroundWorker (legacy)	<code>BackgroundWorker</code> class; <code>DoWork</code> , <code>ProgressChanged</code> , <code>RunWorkerCompleted</code> events; <code>ReportProgress</code> ; <code>CancellationPending</code> ; Cross-thread exception handling; Migration to <code>async/await</code> + <code>IProgress<T></code>	One session max. Teach it because legacy WinForms/WPF code uses it heavily. Emphasize: don't write new code with this.
5	Task Parallel Library (TPL)	<code>Task</code> vs <code>Thread</code> ; <code>Task.Run</code> vs <code>Task.Factory.StartNew</code> ; <code>TaskCreationOptions.LongRunning</code> ; Continuations (<code>ContinueWith</code>); <code>Task.WhenAll</code> , <code>Task.WhenAny</code> , <code>Task.WaitAll</code> ; Cancellation (<code>CancellationToken</code> , <code>CancellationTokenSource</code> , linked tokens); Exception handling; <code>AggregateException</code> ; <code>IProgress<T></code>	Bridge from BackgroundWorker. Show side-by-side equivalents.

#	Phase	Subtopics	Notes
6	async / await	The state machine concept; <code>Task</code>, <code>Task<T></code>, <code>ValueTask<T></code>; <code>SynchronizationContext</code> (and WPF's); <code>ConfigureAwait(false)</code> — when and why; Pitfalls: <code>async void</code>, <code>.Result</code> / <code>.Wait()</code> deadlocks, fire-and-forget done wrong, missing <code>await</code>; Async in constructors / properties / <code>IAsyncDisposable</code>; <code>Task.Delay</code> vs <code>Thread.Sleep</code>	Allocate 2 sessions — this is where interns trip up most. Heavy hands-on.
7	Concurrent Collections	<code>ConcurrentDictionary</code> (and <code>GetOrAdd</code> subtleties); <code>ConcurrentQueue</code>, <code>ConcurrentStack</code>, <code>ConcurrentBag</code>; <code>BlockingCollection<T></code>; Producer/consumer pattern; When a plain <code>List<T> + lock</code> is actually better	Practical, code-heavy session.
8	Parallel Programming	<code>Parallel.For</code>, <code>Parallel.ForEach</code>, <code>Parallel.Invoke</code>; <code>ParallelOptions</code> and degree of parallelism; PLINQ (<code>AsParallel</code>, <code>WithDegreeOfParallelism</code>); Partitioners; When parallelism <i>hurts</i> (small workloads, contention, I/O-bound work)	Show CPU-bound demo with measurable speedup.
9	WPF Threading	Dispatcher and <code>DispatcherObject</code>; <code>Dispatcher.Invoke</code> vs <code>BeginInvoke</code> vs <code>InvokeAsync</code>; <code>DispatcherPriority</code>; <code>DispatcherTimer</code> vs <code>System.Timers.Timer</code> vs <code>System.Threading.Timer</code>; Cross-thread exceptions on UI elements; Async commands in MVVM; Reporting progress to UI with <code>IProgress<T></code>; Freezable objects and freezing for cross-thread use	Essential for your domain. Build a real mini-app.
10	Advanced Patterns	Producer/consumer with <code>Channel<T></code> (<code>System.Threading.Channels</code>); TPL Dataflow basics; <code>Lazy<T></code> thread-safety modes; <code>ThreadLocal<T></code> and <code>[ThreadStatic]</code>; <code>AsyncLocal<T></code>; Async streams (<code>IAsyncEnumerable<T></code>, <code>await foreach</code>)	Optional / opt-in depending on cohort speed.
11	GC & Threading Intersection (<i>short</i>)	GC runs on its own threads (background/concurrent GC); The	30–45 min aside. Relevant to industrial

#	Phase	Subtopics	Notes
		finalizer thread; GC pauses and real-time work; <code>WeakReference</code> , <code>GCHandle</code> in multi-threaded scenarios; Why allocating in hot parallel loops hurts	/ real-time work.
12	Debugging, Diagnostics & Best Practices	VS Threads / Tasks / Parallel Stacks windows; Reproducing race conditions (stress tests, <code>Thread.Yield</code>); Designing thread-safe classes (immutable / externally synced / internally synced); "Async all the way" principle; Decision tree: Thread → ThreadPool → Task → Parallel → async/await → Dataflow/Channels; Performance: lock contention, false sharing, allocation pressure	Capstone session. Combine with code review of intern projects.

Added

Topic
Unsafe Code (<code>unsafe</code> , pointers, <code>fixed</code> , <code>stackalloc</code>)
Full Garbage Collection deep-dive

Teaching Tips

- Run each concept as a small **console demo first**, then move into WPF. Dispatcher pitfalls only surface in a UI app.
- **debug at least one deadlock** by hand — it's the fastest way to internalize lock ordering.
- For async/await, write the **wrong** code first (e.g. `.Result` in a UI handler) and observe the deadlock.